Replace Temp with Query

Refatoração em Python

Deixando o código mais
limpo, claro e reutilizável.

Arthur, Victor e Yong

Análise e Desenvolvimento de Sistemas

2º período

Roteiro da apresentação

O que é a técnica

Entenda o propósito e quando aplicar.

Código sem a técnica

Veja um exemplo do jeito tradicional.

Código com a técnica

Troque a variável por um método.

Comparação e cuidados

Benefícios, trade-offs e pontos de atenção.

Desafio interativo e conclusão

Pratique, revise e leve para o dia a dia.

O que é Replace Temp with Query?

Técnica de refatoração do catálogo de Martin Fowler que substitui uma variável temporária por uma **consulta (método).**

PROBLEMA

Variáveis temporárias guardam um cálculo, mas só existem dentro da função.

SOLUÇÃO

Extrair a expressão para um método que retorna o valor calculado.

RESULTADO

O cálculo ganha nome, pode ser reutilizado e o código fica mais fácil de ler.

Refatorar é melhorar a estrutura do código **sem mudar o que ele faz.**

Como aplicar

4 PASSOS PARA APLICAR A REFATORAÇÃO

01

Verifique que **a variável temporária** recebe valor uma única vez

02

Extraia **a expressão** para um novo método sem efeitos colaterais

03

Substitua cada uso da variável pela chamada do método

04

Remova a variável temporária e rode os testes para confirmar que nada mudou

Código SEM a técnica

A variável temporária fica presa ao método e não pode ser reutilizada.

O cálculo de base está **repetido** em dois métodos

Se a regra mudar, é preciso **alterar em vários lugares**

Métodos mais longos e **difíceis de ler**

```python
class Pedido:
    def __init__(self, quantidade, preco_item):
        self.quantidade = quantidade
        self.preco_item = preco_item

    def preco_final(self):
        base = self.quantidade * self.preco_item
        if base > 1000:
            return base * 0.95
        return base * 0.98

    def resumo(self):
        base = self.quantidade * self.preco_item
        return f'Total bruto: {base}'
```

Código COM a técnica

APÓS A REFATORAÇÃO

O cálculo existe
em **um único lugar**

Qualquer método da classe
pode reutilizar
preco_base()

O nome do método
explica a **intenção**

```
class Pedido:
    def __init__(self, quantidade, preco_item):
        self.quantidade = quantidade
        self.preco_item = preco_item

    def preco_base(self):
        return self.quantidade * self.preco_item

    def preco_final(self):
        if self.preco_base() > 1000:
            return self.preco_base() * 0.95
        return self.preco_base() * 0.98

    def resumo(self):
        return f'Total bruto: {self.preco_base()}'
```

Comparação

Antes e depois: os ganhos da técnica **Replace Temp with Query**

	SEM A TÉCNICA	COM A TÉCNICA
Repetição do cálculo	O cálculo se repete em cada método	O cálculo existe em um só lugar
Manutenção	Mudanças precisam ser feitas em vários pontos	Mudanças são feitas em um único ponto
Legibilidade	É preciso entender a fórmula	O nome do método explica a intenção
Reuso	A variável é local e não pode ser reutilizada	Qualquer método pode usar
Próximas refatorações	Mais difícil aplicar outras técnicas	Mais fácil aplicar técnicas como **Extract Method**

Quando usar e cuidados

Boas práticas e limites da técnica Replace Temp with Query

Use quando

O cálculo é **usado em mais de um lugar**
ou a variável atrapalha outras refatorações.

O valor é calculado **uma única vez**
e não muda depois.

Cuidado quando

O método será executado **a cada chamada**,
o que pode custar desempenho
se o cálculo for pesado.

Não use se a variável recebe **valores diferentes**
ao longo da função.

Dica em Python

Para cálculos simples sem parâmetros, o decorador
@property deixa o uso como **pedido.preco_base**.

Desafio interativo

Responda mentalmente e confira no próximo slide

01 Nesta função, a variável **total** pode virar método?

```
def somar(lista):
    total = 0
    for x in lista:
        total += x
    return total
```

02 Qual o principal benefício de trocar a variável por um método na classe **Pedido**?

Pense. A resposta será revelada no **próximo slide.**

Respostas do desafio

Confira e compare com o que você pensou

01 A variável total pode virar método?

Não, do jeito que está.

total é um acumulador: recebe um novo valor a cada volta do for (total += x). A técnica só vale para variável que recebe valor uma única vez.

Em Python, o mais simples aqui é: return sum(lista)

02 Principal benefício na classe Pedido

O cálculo passa a existir em um único lugar e pode ser reutilizado por qualquer método da classe.

Menos repetição, manutenção em um só ponto e um nome que explica a intenção.

Conclusão e perguntas?

Replace Temp with Query troca variável temporária por método

Menos repetição, mais reuso e código mais legível

Use com atenção ao desempenho e a variáveis que mudam de valor

Referência: Martin Fowler, Refactoring

Obrigado!
Perguntas?